

Da Página Parada à Página Viva

Conectando HTML/CSS (Semana 07) com DOM e JavaScript (Semana 08)

Desenvolvimento Front-End [React] T4 | Módulo 1 | Semana 8 | Professor: Lisandro Faria Pinheiro

Este documento não ensina conteúdo novo — ele existe para um único objetivo: mostrar ao aluno que ele **não está começando do zero**. Tudo que foi aprendido na Semana 07 (HTML e CSS — Parte 1) é a base direta de tudo que será visto na Semana 08 (DOM e JavaScript). O aluno já sabe mais do que imagina.

1. O que vocês já sabem (recapitulando a Semana 07)

Na semana passada, "HTML e CSS — Parte 1", vocês construíram páginas **estáticas**: escreveram a estrutura em HTML e definiram a aparência em CSS.

- **Introdução às linguagens HTML e CSS** — o que cada uma faz e por que trabalham juntas.
- **Introdução e estrutura do HTML** — tags, atributos, `id`, `class`, hierarquia de elementos.
- **Introdução e estrutura do CSS** — seletores (`#id`, `.classe`, `tag`), regras, propriedades e valores.
- **Alterando estilos e propriedades** — `color`, `background-color`, `font-size`, `border-radius`, e como aplicar essas propriedades a um ou vários elementos usando seletores.

Se vocês conseguem escrever isto...

```
<h1 id="titulo" class="destaque">Bem-vindo!</h1>
```

```
#titulo {  
  color: #0d3b66;  
}  
.destaque {  
  background-color: yellow;  
  font-weight: bold;  
}
```

...então vocês **já sabem** o que é um `id`, o que é uma `class`, como um seletor `#` aponta para um `id` e um seletor `.` aponta para uma `class`, e como uma propriedade CSS recebe um valor. Essa é exatamente a base da Semana 08.

2. O que muda a partir de agora

Na semana passada, depois que a página carregava, ela **não mudava mais**. O HTML e o CSS descrevem um estado fixo: "o título é azul", "o card tem fundo amarelo". Ponto final.

A partir desta semana, a página passa a **responder**: o aluno vai aprender a fazer o título mudar de cor quando alguém clica, a lista crescer quando alguém digita, a caixa se mover quando alguém aperta uma tecla. O nome técnico para essa virada de chave é **DOM (Document Object Model) + JavaScript**.

A ideia central desta semana, em uma frase: o navegador lê o HTML que vocês escreveram e cria, na memória, uma cópia viva dele — o DOM. O JavaScript não conversa com o arquivo `.html` salvo no disco; ele conversa com

essa cópia viva. E é por isso que a página pode mudar na tela sem que o arquivo original mude uma vírgula.

3. A analogia central: o corpo que ganha vida

Para fixar essa conexão, use a analogia do **corpo humano**. Ela conecta as quatro tecnologias da mesma linha de raciocínio, em vez de apresentá-las como quatro matérias soltas.

Camada	Tecnologia	Papel no corpo	O que ela decide
1	HTML	O esqueleto	Onde fica cada osso — a estrutura, os elementos, a hierarquia
2	CSS	A pele, o rosto, as roupas	A aparência — cor, tamanho, espaçamento, forma
3	DOM	O sistema nervoso	A ligação entre o corpo e o cérebro — permite "sentir" e "comandar" cada parte
4	JavaScript	O cérebro e os músculos	A decisão e a ação — o que fazer quando algo acontece (um clique, uma tecla)

Na Semana 07, vocês montaram o esqueleto (HTML) e vestiram o corpo (CSS). O corpo já existia, já tinha aparência — mas estava parado, como um manequim. Nesta semana, o navegador dá a esse manequim um sistema nervoso automaticamente (o DOM, criado a partir do próprio HTML de vocês) e vocês vão aprender a escrever o cérebro (JavaScript) que decide como esse corpo reage ao mundo.

Importante: o DOM não é uma tecnologia nova e separada para decorar do zero. É o **mesmo HTML** que vocês já escrevem, só que "vivo" na memória do navegador em vez de parado no arquivo.

4. A lógica do DOM: navegando na tela a partir da raiz HTML

A Seção 3 chamou o DOM de "sistema nervoso". Falta entender uma peça central desse sistema: ele não é uma lista solta de elementos — é uma **árvore**, com um único ponto de partida. Retomando o exemplo da Seção 1, o navegador não guarda `<h1 id="titulo" class="destaque">Bem-vindo!</h1>` do jeito que foi digitado. Ele monta uma árvore de objetos assim:

```
document
├── html
│   └── body
│       └── h1#titulo.destaque
│           └── "Bem-vindo!"
```

`document` é a **raiz** dessa árvore — é a partir dele que todo o JavaScript da página começa a navegar. Um comando como `document.querySelector("#titulo")` não "procura" o elemento em algum arquivo: ele desce pela árvore, a partir da raiz, até achar o nó que combina com o seletor, e devolve uma **referência a esse nó exato** — não uma cópia, o próprio objeto que está na tela.

É por isso que isto funciona:

```
const titulo = document.querySelector("#titulo");
titulo.textContent = "Olá, turma!";
```

A variável `titulo` não guarda um texto: guarda um **ponteiro** para aquele nó específico da árvore. Quando alteramos `titulo.textContent`, não reescrevemos o arquivo `.html` — alteramos o objeto que está na memória do navegador, e o navegador redesenha só aquele trecho da tela.

Essa navegação fica ainda mais clara quando existe mais de um elemento com a mesma estrutura, como os cards de um catálogo: `document.querySelectorAll(".card")` não devolve um texto, devolve uma **lista de referências**, uma para cada nó `.card` daquela árvore — cada uma pode ser lida, alterada ou removida individualmente, sem afetar as outras.

Da raiz até a reação na tela: a árvore, sozinha, é estática — ela só passa a "reagir" quando um evento entra em cena. O caminho do clique do usuário até a tela mudar segue sempre a mesma ordem:

```
Usuário interage (clique, tecla, envio de formulário)
↓
O evento é disparado no nó correspondente da árvore
↓
addEventListener executa a função registrada
↓
A função altera um ou mais nós do DOM (textContent, classList, style...)
↓
O navegador redesenha apenas os nós alterados
```

Nenhuma etapa desse fluxo recarrega a página. O arquivo `.html` salvo em disco nunca é tocado — tudo acontece na árvore que existe na memória do navegador.

E quando um nó deixa de existir? A mesma lógica vale ao contrário. Quando um elemento é removido da árvore com `elemento.remove()`, ele deixa de aparecer na tela. Se nenhuma variável do código ainda guardar uma referência a esse nó, o navegador libera automaticamente o espaço que ele ocupava na memória — um processo chamado **Garbage Collector** (coletor de lixo). O aluno não precisa administrar essa limpeza manualmente: ela acontece porque o nó, sem nenhum galho da árvore e sem nenhuma variável apontando para ele, se torna inacessível.

Voltando à analogia da Seção 3: navegar o DOM é a forma como o cérebro (JavaScript) encontra um nervo específico (um nó) dentro do sistema nervoso (o DOM) para comandá-lo — sempre partindo da coluna vertebral, o `document`.

5. Tabela de conexão direta: o que muda de nome, o que continua igual

Esta é a parte mais importante deste documento. Compare, linha a linha, algo que vocês já fizeram na Semana 07 com a forma equivalente na Semana 08.

Semana 07 (HTML/CSS) — o que vocês já sabem	Semana 08 (DOM/JS) — a mesma ideia, ativada por código	O que realmente mudou
<code><h1 id="titulo"></code> no HTML	<code>document.getElementById("titulo")</code>	Nada de novo no <code>id</code> — agora ele é "capturado" numa variável
Seletor CSS <code>#status</code> , <code>.card</code> , <code>div > p</code>	<code>document.querySelector("#status")</code> , <code>document.querySelector(".card")</code>	A MESMA sintaxe de seletor CSS que vocês já usam no <code>.css</code>
Uma regra CSS que vale para várias classes (<code>.item { ... }</code>)	<code>document.querySelectorAll(".item")</code> percorre todos os <code>.item</code>	O "todos os elementos com esta classe" que o CSS já aplicava sozinho, agora é uma lista que dá pra percorrer
<code>color: red;</code> escrito direto no arquivo <code>.css</code>	<code>elemento.style.color = "red"</code> no JavaScript	Mesma propriedade CSS; só passa a poder mudar durante o uso da página
<code>background-color:</code> <code>yellow;</code> no CSS	<code>elemento.style.backgroundColor =</code> <code>"yellow"</code> no JS	Mesmo nome, só troca o hífen por camelCase (<code>backgroundColor</code>)
Uma classe <code>.destaque</code> definida no <code>.css</code>	<code>elemento.classList.add("destaque")</code> / <code>.toggle("destaque")</code>	O estilo continua 100% definido no CSS — o JS só liga/desliga a classe
Texto fixo escrito direto entre as tags (<code><p>Olá</p></code>)	<code>elemento.textContent = "Olá"</code>	O mesmo texto, agora pode ser trocado depois que a página já carregou
A página sempre abre do mesmo jeito	<code>elemento.addEventListener("click", ...)</code>	faz algo acontecer só quando o usuário agir — a novidade real da semana: tempo e interação

Conclusão prática desta tabela: a maior parte da "sintaxe nova" desta semana não é conhecimento novo — é o **mesmo conhecimento de seletores e propriedades CSS**, só que chamado de dentro do JavaScript. O que é genuinamente novo é a ideia de **reagir a um evento**.

6. Exemplo prático: o mesmo projeto, um passo além

Imagine que, na Semana 07, um aluno entregou este HTML e CSS:

```
<h1 id="titulo">Minha Loja</h1>
<button id="botao">Ver oferta</button>
```

```
#titulo {  
  color: #0d3b66;  
}  
.destaque {  
  color: #e76f51;  
  font-weight: bold;  
}
```

Nada nesse projeto muda sozinho: o `#titulo` é sempre azul, e a classe `.destaque` só existe se alguém escrever `class="destaque"` direto no HTML.

Agora, com três linhas de JavaScript, o mesmo HTML e o mesmo CSS ganham vida — sem que uma única linha do CSS precise mudar:

```
// 1) Selecionamos o botão que já existe no HTML (Semana 07)  
const botao = document.getElementById("botao");  
const titulo = document.getElementById("titulo");  
  
// 2) Registramos uma reação a um evento (a novidade da Semana 08)  
botao.addEventListener("click", function () {  
  // 3) Ativamos a MESMA classe .destaque que já existia no CSS da Semana 07  
  titulo.classList.toggle("destaque");  
});
```

O CSS de vocês nunca soube que ia ser "ativado por um clique" — e não precisa saber. Ele só define como as coisas devem parecer. O JavaScript decide **quando** aplicar isso.

7. Roteiro sugerido para abrir o Encontro 1 (15/09)

Sugestão de fala para os primeiros minutos da aula, retomando a semana anterior antes de introduzir o DOM:

*"Na semana passada vocês construíram páginas com HTML e CSS. Levantem a mão quem usou um `id` no HTML... quem usou uma `class` ... quem escreveu `color` ou `background-color` no CSS. Ótimo — vocês já sabem 80% do que precisam para esta semana. A única coisa que falta é aprender a **chamar** esse `id`, essa `class` e essas propriedades de dentro do JavaScript, e a fazer isso acontecer **quando o usuário fizer alguma coisa** — um clique, uma tecla. É exatamente isso que vamos fazer nos próximos três encontros."*

Depois dessa fala, siga com o Bloco 1 do Guia do Professor (o que é o DOM).

8. Erros de mentalidade para evitar

- **"Vou ter que aprender seletores de novo, do zero."** Não. `#id`, `.classe` e `tag` em `querySelector` são exatamente os mesmos seletores do CSS.
- **"O estilo em JavaScript é uma linguagem totalmente diferente."** Não é. É a mesma propriedade CSS, só que escrita em camelCase (`font-size` vira `fontSize`).
- **"O DOM é uma quarta linguagem que eu preciso decorar."** Não é uma linguagem — é a representação em memória do HTML que o próprio aluno já escreve. Não existe "sintaxe do DOM" para decorar; existe a forma de acessá-lo a partir do JavaScript.

- **"Agora o CSS não serve mais para nada, é tudo no JavaScript."** Ao contrário: o CSS continua sendo o lugar certo para definir a aparência (`classList` é preferível a `style` exatamente por isso). O JavaScript só liga e desliga o que o CSS já definiu.

9. Checklist mental antes de começar a Semana 08

Antes de cada novo conceito desta semana, pergunte: **"isso já existia no HTML/CSS da semana passada, ou é realmente novo?"**

- `id` e `class` → já existia (Semana 07). Só ganha uma forma de ser "prego" em JS.
- Seletores `#`, `.`, `tag` → já existia (Semana 07, no `.css`). Mesma sintaxe em `querySelector`.
- Propriedades CSS (`color`, `background-color` ...) → já existia (Semana 07). Só troca hífen por camelCase.
- Estrutura em árvore, pai/filho/irmão → já existia implicitamente (é assim que o HTML sempre foi aninhado). A novidade é dar nome técnico a isso (DOM) e poder navegar por código.
- Reagir a clique, digitação, mouse → isto sim é novo. É a única peça genuinamente inédita da semana — tudo o resto é o conhecimento da Semana 07 sendo reaproveitado.

10. Mapa-resumo da conexão

```
SEMANA 07                                SEMANA 08
-----                                -----
HTML (estrutura, id, class)  ----> DOM (a mesma estrutura, viva na memória)
CSS (seletores e propriedades) -> querySelector / style / classList
                                (os MESMOS seletores e propriedades,
                                agora chamados de dentro do JS)

                                + addEventListener
                                (a peça nova: reagir a uma ação
                                do usuário, em tempo real)
```

Resultado: a página que antes era um retrato parado passa a ser um vídeo que responde a quem assiste.